

Broken Authentication

Vulnerability 1 — Login Brute Force

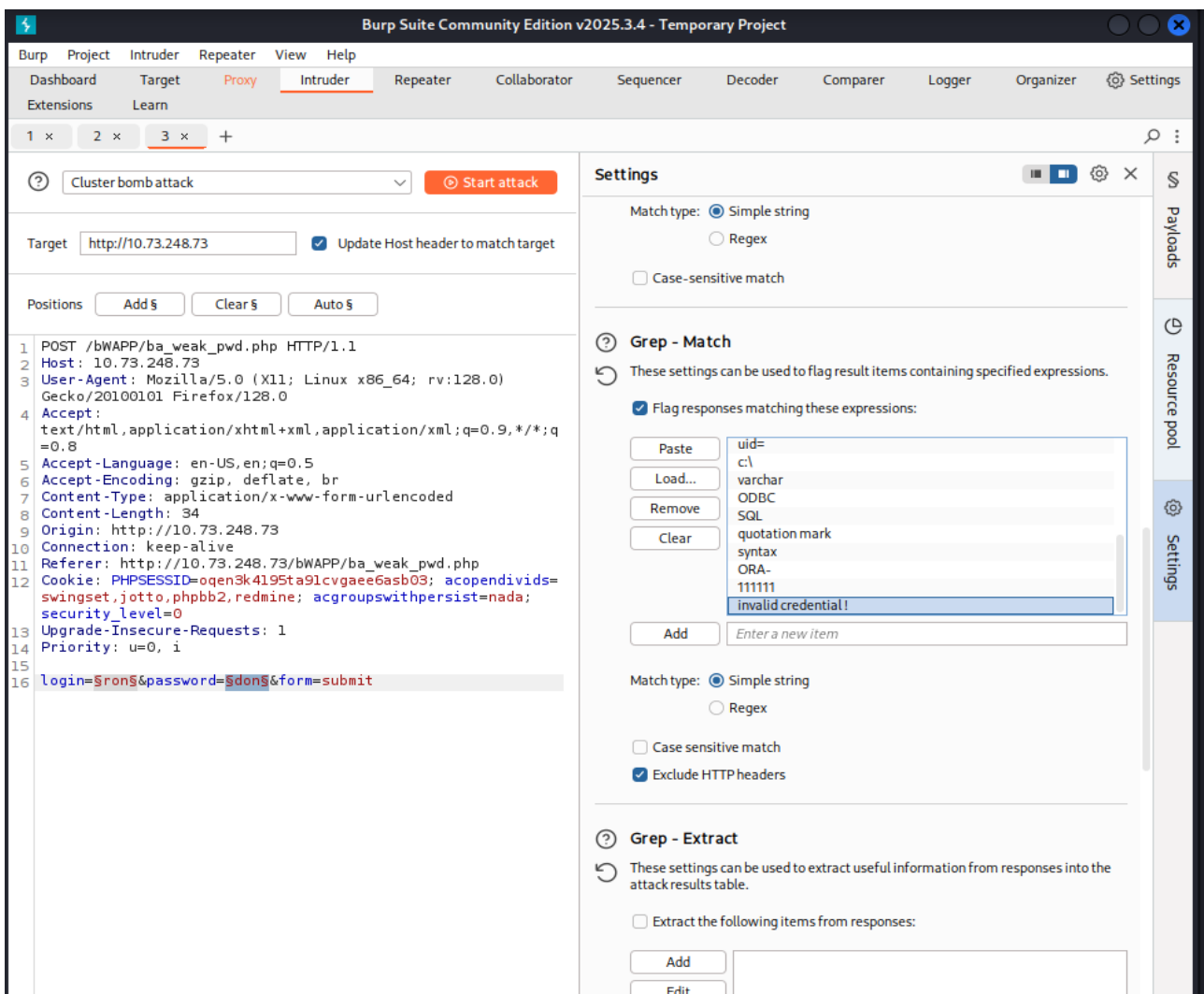
Description:

Weak authentication and no account lockout allow attackers to guess credentials using automated tools.

Tools Used: Burp Suite Intruder

Exploitation Steps:

1. Intercept login request using Burp Proxy.
2. Send request to Intruder and mark `login=` and `password=` as payload positions.
3. Load common username and password wordlists.
4. Set attack type: **Cluster Bomb**.
5. Start the attack and observe responses; successful login shows change in status/response length.



Vulnerability 2 — Session Fixation / Hijacking

Description:

The application relies only on the session ID for authentication, allowing attackers to reuse another user's session cookie.

Tools Used: Burp Suite Proxy / Repeater

Exploitation Steps:

1. Log in as a valid user and intercept the request to capture the session cookie (PHPSESSID).
2. Copy the cookie value.
3. Paste the stolen cookie in the browser of another user account (modify via storage/cookie editor or proxy).
4. Refresh page — the second user becomes logged in as the first user.

The image shows a screenshot of a web browser displaying the DVWA (Damn Vulnerable Web Application) interface. The browser's address bar shows the URL `10.73.248.73/dvwa/setup.php`. The page title is "Database setup". The page content includes instructions for creating or resetting the database, a "Create / Reset Database" button, and a "Backend Database: MySQL" label. The page also displays a sidebar with navigation links: Home, Instructions, Setup, Bruteforce, Command Execution, CSRF, Insecure CAPTCHA, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The bottom of the page shows the username "user", security level "low", and PHPIDS status "enabled".

On the left side of the image, the Burp Suite interface is visible. The "Repeater" tab is active, showing a list of requests. The first request is a GET request to `/dvwa/setup.php`. The request details are displayed in the "Pretty" view, showing the following headers and body:

```
1 GET /dvwa/setup.php HTTP/1.1
2 Host: 10.73.248.73
3 Cache-Control: max-age=0
4 Accept-Language: en-US,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Referer: http://10.73.248.73/dvwa/
9 Accept-Encoding: gzip, deflate, br
10 Cookie: security=low; acpendsid=wingset,jotto,phpbb2,redmine; acgroupswithpersist=nada; PHPSESSID=7349hgl6k8710idparsufukn2; security_level=0
```

The image shows a web browser displaying the DVWA Security page and a Burp Suite proxy history window.

DVWA Security Page:

- Navigation menu: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, Insecure CAPTCHA, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored.
- DVWA Security** (locked icon):
 - Script Security: Security Level is currently **low**. You can set the security level to low, medium or high. The security level changes the vulnerability level of DVWA. (low) [Submit]
 - PHPIDS**: PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP. You can enable PHPIDS across this site for the duration of your session. PHPIDS is currently **enabled**. [disable PHPIDS] [Simulate attack] - [View IDS log]
- Footer: Username: admin, Security Level: low, PHPIDS: enabled. Damn Vulnerable Web Application (DVWA) v1.8

Burp Suite Proxy History:

Time	Type	Direction	Method	URL
13:28:5...	HT...	→ Request	GET	http://10.73.248.73/dvwa/security.php
13:29:0...	HT...	→ Request	GET	http://10.73.248.73/dvwa/setup.php

Request Details:

```
1 GET /dvwa/security.php HTTP/1.1
2 Host: 10.73.248.73
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
  Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://10.73.248.73/dvwa/security.php
8 Connection: keep-alive
9 Cookie: security=low; security_level=0; PHPSESSID=7349hg10k07i0idpursufjkn2;
  acpendsivids=swingset,jotto.phpbb2,redmine; acgroupswithpersist=nada
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13
```

I copy the session id from one browser by intercepting in burp then I Pasted that id into other browser and I was able to log in as admin in that browser .

Vulnerability 3 — Privilege Escalation via Cookie Modification

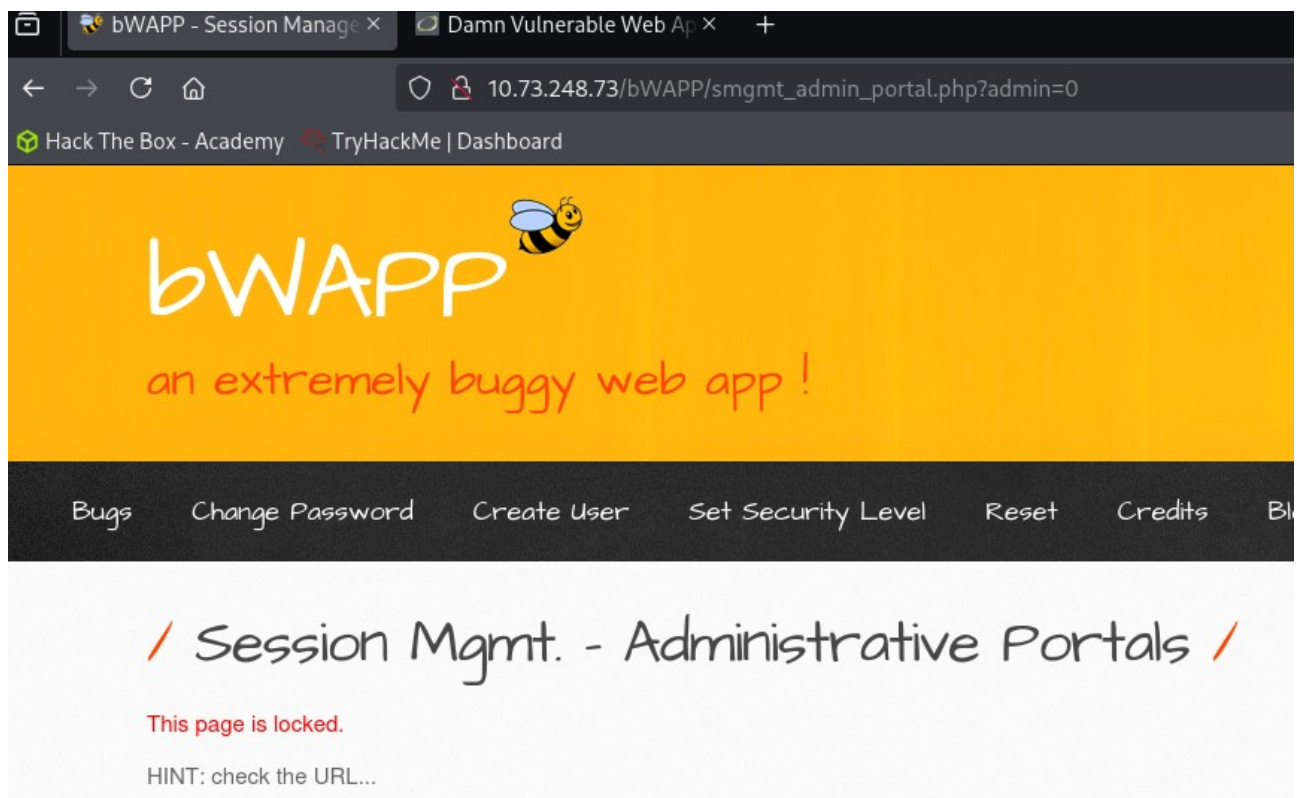
Description:

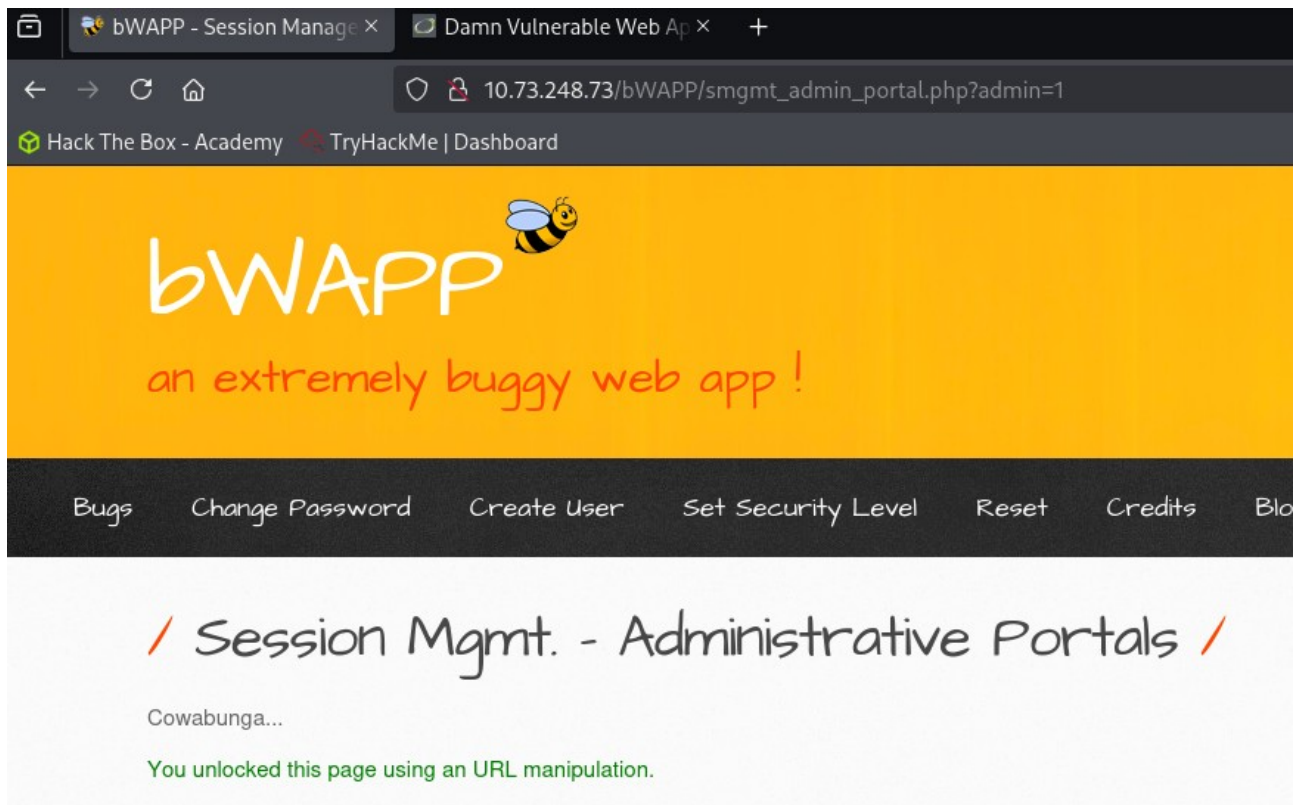
Authorization depends on a cookie parameter instead of server-side role validation.

Tools Used: Browser + Burp

Exploitation Steps:

1. Navigate to a normal user session.
2. Capture cookies for the request.
3. Locate role-related parameter (example: `admin=0` or role field).
4. Modify it (e.g., `admin=1`) and forward the request/refresh the page.
5. Access to **admin portal** is granted without proper permissions.





Now I succesfully logged in by changing the admin=1